



Projet DEEP

Rapport intermédiaire – Tictac

Thomas PINEAU | Milovan PAGET-MIOT—SALLES

Table des matières

Introduction	2
Cahier des charges	2
Description de la montre	2
Interactions utilisateur	3
Limites et contraintes du projet	4
Limites fonctionnelles	4
Contraintes techniques	4
Composants utilisés	5
Tableau de l'affectation des ports du microcontrôleur	6
Cahier de suivi	7
État d'avancement et analyse du projet réalisé	8
État actuel du projet	8
Hardware	8
Software	9
Points restant à finaliser	9
Hardware	9
Software	9
Documentation	10
Analyse critique et améliorations envisagées	10
Hardware	10
Software	10
Bilan des objectifs atteints	11
Software	11
Interface graphique et écran e-Ink	11
Adaptation d'une police d'écriture pour microcontrôleur	11
Conversion d'icône en format bitmap	12
Design d'interface utilisateur avec Figma	12
Traitement des données de l'accéléromètre	13
Acquisition des données de l'accéléromètre	13
Visualisation des axes bruts de l'accéléromètre	13
Norme de l'accélération non filtrée	14
Réponse en fréquence du filtre passe-bande	15
Diagramme pôle-zéro du filtre	15
Signal filtré et détection des pas	16
Spectre de Fourier du signal filtré	16
Essentiel à retenir pour le podomètre sur MATLAB	17
Conception assistée par ordinateur d'un châssis (CAO)	17
Compléments envisagés	18

Introduction

Tictac est un prototype de montre connectée développée sur microcontrôleur STM32G4. Ce dispositif combine les fonctionnalités d'une montre classique (affichage de l'heure, alarme programmable) avec des capacités de suivi de santé et d'activité physique, tout en intégrant des fonctionnalités modernes comme l'émulation NFC, dans un design économe en énergie, minimaliste et intuitif.

Cahier des charges

Description de la montre

- **Heure** – La montre permettra d'afficher l'heure avec le moins de retard possible, en reposant sur l'horloge temps réel du microcontrôleur. L'horloge pourra être synchronisée avec un serveur temporel, automatiquement par intervalle défini ou manuellement par l'utilisateur, grâce à une connexion Bluetooth entre la montre et un smartphone, via une application dédiée.
- **Alarme** – L'utilisateur pourra configurer une alarme en utilisant l'interface prévue à cet effet, en reposant sur l'intégration d'un buzzer. L'utilisateur pourra sélectionner l'heure de déclenchement de l'alarme ainsi que la tonalité utilisée parmi une sélection de tonalités prédéfinies dans le programme.
- **Affichage** – La montre intègre une interface graphique simple et facile de compréhension pour l'utilisateur, sur une surface d'écran restreinte (1.54 pouces) et à taux de rafraîchissement bas (2 secondes par rafraîchissement complet). Cette interface sera contrôlée avec 3 boutons physiques (boutons gauche, central et droit). La montre devra être le plus économe en énergie, en intégrant un écran à encre électronique (écran e-Ink).
- **Mesure de pouls** – La montre permettra la mesure du pouls de l'utilisateur en intégrant un capteur de fréquence cardiaque. La montre permettra également la conservation des données mesurées dans un menu dédié à l'historique des mesures, tant que la montre ne sera pas mise hors tension.
- **Podomètre** – La montre effectuera la mesure du nombre de pas parcourus par l'utilisateur au cours d'une journée complète en intégrant un accéléromètre. La longueur d'un pas est estimée en fonction de la taille de l'utilisateur. La mesure du nombre de pas parcourus permettra à un algorithme de calculer la distance parcourue en kilomètres au cours d'une journée complète. Les mesures de distance et de temps permettront de déterminer la vitesse de l'utilisateur par intervalle défini.
- **NFC** – La montre permettra la lecture, la duplication et l'émulation de badges NFC en intégrant un émetteur-récepteur NFC/RFID.

Interactions utilisateur

- **Consultation de l'heure** – L'utilisateur voit immédiatement l'heure au format 24h sur l'écran, avec la date et le nombre de pas effectués durant la journée, sur l'écran de veille.
- **Navigation dans l'interface** – L'utilisateur navigue à travers l'interface et les différents menus en utilisant les boutons gauche et droit, et utilise le bouton central pour la sélection ou la validation.
- **Menu principal** – L'utilisateur peut accéder aux différentes applications via le menu principal, en appuyant sur le bouton central à partir de l'écran de veille. Il navigue à travers ce menu en utilisant les boutons gauche et droit, puis sélectionne une application en utilisant le bouton central.
- **Suivi d'activité** – L'utilisateur peut consulter son activité (nombre de pas journalier, distance parcourue et vitesse moyenne) ou peut accéder à l'historique de l'activité effectuée au cours des derniers jours (sous réserve de la disponibilité des données en mémoire).
- **Mesure du rythme cardiaque** – L'utilisateur peut démarrer une mesure de son rythme cardiaque et est guidé à travers une série d'interfaces graphiques tout au cours de la procédure. Il peut également consulter l'historique des mesures de son rythme cardiaque effectuées au cours des derniers jours (sous réserve de la disponibilité des données en mémoire).
- **Configuration d'une alarme** – L'utilisateur peut configurer une alarme à travers le menu dédié à cet effet. Il peut sélectionner l'heure à laquelle l'alarme se déclenchera, et la tonalité utilisée parmi une sélection de tonalités prédéfinies dans le programme.
- **Fonctionnalités NFC** – L'utilisateur peut lire, dupliquer et émuler des badges NFC. Il devra approcher la montre du badge ou du lecteur cible, et sera guidé à travers une série d'interfaces graphiques tout au long des différentes procédures.
- **Synchronisation avec un smartphone** – L'utilisateur peut synchroniser la montre avec un smartphone, à travers une application dédiée à cet effet. Il peut activer la synchronisation depuis le menu paramètres, puis configurer la synchronisation de la date et de l'heure ainsi que de l'historique des données mesurées.

Limites et contraintes du projet

Limites fonctionnelles

Les données de mesure des capteurs d'activité physique (distance parcourue, pas et relevés cardiaques) ne sont pas conservées sur la montre après la mise hors tension de la montre (pas de stockage permanent des données).

La montre se limite à la mesure de la fréquence cardiaque en raison du modèle de capteur utilisé : pas d'oxymétrie, d'électrocardiogramme ou d'autres fonctionnalités.

La fonction GPS n'est pas intégrée à la montre : la distance parcourue est calculée uniquement à partir du nombre de pas et de la taille de l'utilisateur définie par ce dernier dans les paramètres de la montre.

Contraintes techniques

L'écran e-Ink de la montre est à faible taux de rafraîchissement : il se rafraîchit complètement toutes les 2 secondes : la mise en place d'animations n'est pas possible. La surface d'affichage de 1.54 pouces nécessite une interface minimaliste et des informations condensées. L'affichage est monochrome et l'écran n'est pas tactile : la navigation devra se faire via des boutons physiques.

La synchronisation de l'heure avec un serveur temporel, le réglage des informations utilisateur (âge, taille) et la synchronisation de l'historique des données mesurées nécessitent une connexion avec l'application du smartphone.

La montre ne dispose pas de connectivité Internet intégrée : tout échange de données avec un système externe passe par la connexion au smartphone via une application dédiée à cet effet.

Composants utilisés

Nom ou référence du composant	Description / Rôle dans l'application / Caractéristiques principales dans le projet
Écran e-paper – epd1in54_V2	Écran carré de 1,54 pouces, monochrome, résolution 200x200 pixels, communiquant via SPI, fonctionnant sous 3,3 V. Il est piloté à l'aide des bibliothèques fournies par le constructeur (voir annexe) et sert à l'affichage de l'interface graphique de la montre. Son principal avantage réside dans sa très faible consommation d'énergie : 5 μ A en veille profonde, 50 μ A en veille.
Buzzer	Dispositif à deux broches commandé par PWM, utilisé pour signaler l'échéance d'une alarme programmée par l'utilisateur.
Accéléromètre – MPU6050	Capteur 6 axes combinant accéléromètre et gyroscope, communiquant via I ² C, fonctionnant sous 3,3 V. Il est utilisé pour détecter les mouvements de la montre, notamment pour le comptage de pas et de distance parcourue. Les bibliothèques du BSP facilitent son intégration.
Capteur de pouls – HW827	Capteur optique de fréquence cardiaque utilisant la photo-pléthysmographie. Il émet une lumière verte (~525 nm) pour détecter les variations du flux sanguin au poignet. Il communique via I ² C et fonctionne sous 3,3 V.
Boutons poussoirs (gauche, central, droit)	Ils sont utilisés pour l'interaction avec l'interface utilisateur. Connectés à des entrées numériques avec résistances de pull-down, ils permettent la navigation dans les menus et la validation d'actions.
Transmetteur NFC – PN532	Module de communication sans fil NFC/RFID. Il permet la lecture et l'écriture de tags NFC, ainsi que la gestion des cartes d'identification ou des communications entre appareils. Il prend en charge les interfaces I ² C, SPI et UART et fonctionne sous 3,3 V.

Tableau de l'affectation des ports du microcontrôleur

Pin	Commentaires
PA9	Transmetteur NFC – PN532 – [GPIO RST]
PA10	Transmetteur NFC – PN532 – [SPI MISO]
NRST	
GND	
PA12	Bouton poussoir (gauche) – [GPIO]
PB0	Écran e-paper – epdiin54_V2 – [GPIO BUSY]
PB7	Accéléromètre MEMS MPU6050 – [I2C1 SDA]
PA15	Accéléromètre MEMS MPU6050 – [I2C1 SCL]
PB6	
PF0	
PF1	
PA8	Écran e-paper – epdiin54_V2 – [GPIO DC]
PA11	Bouton poussoir (central) – [GPIO]
PB5	Bouton poussoir (droit) – [GPIO]
PB4	
VIN	
GND	
NRST	
5V	
PA2	(Bluetooth) – HCo5 – [UART2 RX] (Debug)
PA7	Écran e-paper – epdiin54_V2, Transmetteur NFC – PN532 – [DIN/MOSI]
PA6	Transmetteur NFC – PN532 – [GPIO SS]
PA5	Écran e-paper – epdiin54_V2, Transmetteur NFC – PN532 – [CLK]
PA4	Écran e-paper – epdiin54_V2 – [CS]
PA3	(Bluetooth) – HCo5 – [UART2 TX] (Debug)
PA1	Buzzer – [GPIO]
PA0	Capteur de pouls – HW-827 – [ADC1_IN1]
AVDD	
3V3	
PB3	Écran e-paper – epdiin54_V2 – [GPIO RST]
PB8	(non dispo)

Cahier de suivi

Date	Tâches, réalisateurs, difficultés rencontrées	A faire la prochaine fois
12/03/2025	Thomas & Milovan : Recherche du sujet, description des fonctionnalités à intégrer (cahier des charges), choix du DEEP PCB	Peaufiner le sujet, préciser la description des fonctionnalités à intégrer et le cahier des charges, commencer à câbler le PCB et effectuer des recherches sur un module
19/03/2025	Thomas & Milovan : Description des fonctionnalités à intégrer, suite du cahier des charges, rédaction du schéma hardware Thomas : Câblage du PCB Milovan : Recherches sur l'écran e-Ink (software, fonctionnement) Difficultés rencontrées : Atelier de câblage bondé	Continuer le câblage du PCB, commencer à réaliser le schéma Altium et continuer les recherches sur un autre module du projet
26/03/2025	Thomas : Réalisation du schéma Altium, câblage du PCB Milovan : Recherches sur le capteur de pouls (software, visualisation à l'oscilloscope)	Continuer le schéma Altium, commencer les tests unitaires sur l'écran e-Ink
28/03/2025	Thomas : Réalisation du schéma Altium Milovan : Adaptation de bibliothèque pour l'écran e-Ink Difficultés rencontrées : La bibliothèque fournie pour l'écran e-Ink n'est pas compatible avec le modèle qui nous a été fourni	Continuer le schéma Altium, continuer l'adaptation de bibliothèque pour l'écran e-Ink
02/04/2025	Thomas : Réalisation du schéma Altium, recherches sur l'accéléromètre Milovan : Adaptation de bibliothèque pour l'écran e-Ink	Continuer le schéma Altium, continuer l'adaptation de bibliothèque pour l'écran e-Ink, commencer la réalisation d'interfaces utilisateur (UI) et finaliser les recherches sur l'accéléromètre
04/04/2025	Thomas & Milovan : Recherches sur l'accéléromètre Thomas : Réalisation du schéma Altium Milovan : Adaptation de bibliothèque pour l'écran e-Ink, création d'interfaces utilisateur (UI), conception d'un châssis pour la montre	Finaliser le schéma Altium, continuer l'adaptation de bibliothèque pour l'écran e-Ink, continuer la réalisation d'UI et

		continuer la conception du châssis
09/04/2025	Thomas : Réalisation du schéma Altium, Élaboration du code d'acquisition des données pour l'accéléromètre en C Milovan : Adaptation de bibliothèque pour l'écran e-Ink, création d'interfaces utilisateur, conception d'un châssis pour la montre	Commencer la rédaction du rapport intermédiaire, commencer la programmation de l'accéléromètre, continuer la réalisation d'UI et la conception du châssis
11/04/2025	Thomas : Recherches sur l'accéléromètre et création d'algorithme pour détecter le nombre de pas effectués (MATLAB) Milovan : Création d'interfaces utilisateur, rédaction du rapport intermédiaire, conception d'un châssis pour la montre	Continuer la réalisation de l'algorithme, continuer la réalisation d'UI et la conception du châssis, continuer la rédaction du rapport intermédiaire
23/04/2025	Thomas : Création d'algorithme pour détecter le nombre de pas effectués (MATLAB), élaboration du code d'acquisition des données pour le capteur de pouls, conversion du code MATLAB vers une cible en C embarqué via une application MATLAB Milovan : Création d'interfaces utilisateur, rédaction du rapport intermédiaire, conception d'un châssis pour la montre	Terminer la rédaction du rapport intermédiaire
30/04/2025	Thomas : Rédaction du rapport intermédiaire Milovan : Rédaction du rapport intermédiaire, conception d'un châssis pour la montre	Continuer la réalisation de l'algorithme, validation et test du code pour le module podomètre. Continuer la réalisation d'UI et la conception du châssis

État d'avancement et analyse du projet réalisé

État actuel du projet

À ce stade du projet, nous avons réalisé les avancées suivantes :

Hardware

- **Câblage du PCB** : Réalisation du câblage du DEEP PCB avec les connexions des différents composants.

- **Schéma Altium** : Réalisation du schéma électronique incluant tous les composants (écran e-Ink, accéléromètre, capteur de pouls, boutons, transmetteur NFC et buzzer).
- **Prototype de châssis** : Conception et impression 3D d'un châssis en plusieurs itérations (pour le moment 8 versions), avec fixations pour le bracelet.

Software

- **Interface graphique**
 - Adaptation d'une bibliothèque compatible avec le modèle d'écran e-Ink dont nous disposons (trouvée sur le site du constructeur)
 - Implémentation de la police Pixeloid Sans en 9 tailles différentes
 - Conversion de 10 icônes en format bitmap exploitable par le microcontrôleur
 - Conception des différentes interfaces utilisateur sur Figma
- **Algorithmes**
 - Développement d'un algorithme de détection de pas utilisant le traitement du signal sur MATLAB
 - Analyse des données d'accéléromètre avec filtrage et détection de pics
 - Adaptation de l'algorithme de détection de pas pour la détection de la fréquence cardiaque

Points restant à finaliser

Hardware

- **Finalisation du routage PCB** : Optimisation des connexions entre les composants
- **Intégration physique de l'écran et du bracelet** : Assemblage final des composants dans la version finale du châssis
- **Finalisation du prototype de châssis** : Optimisation pour l'impression 3D et ajout des emplacements pour les boutons, polissage du châssis

Software

- **Implémentation des algorithmes sur le microcontrôleur** : Transposition des algorithmes MATLAB vers du code C exécutable sur le STM32
- **Communication NFC** : Développement du module de lecture/émulation NFC
- **Connexion Bluetooth** : Développement de la synchronisation avec smartphone
- **Intégration globale** : Fusion des différents modules logiciels (interface, capteurs et traitement des données)

Documentation

- **Documentation Doxygen** : Mise en place de la documentation du code source
- **Réalisation d'une courte vidéo de présentation**
- **Réalisation de la soutenance finale**
- **Réalisation du rapport final**

Analyse critique et améliorations envisagées

Hardware

- **Choix des composants** : Le capteur de pouls HW-827 est limité en précision et ne permet pas une mesure directement sur le poignet, mais uniquement sur le bout du doigt, ce qui n'est pas réellement adapté à une intégration dans une montre. Un modèle plus précis et adapté aux mesures sur le poignet serait préférable pour un projet de ce type.

Software

- **Algorithmes embarqués** : L'adaptation des algorithmes de traitement des données MATLAB au microcontrôleur risque de représenter un certain défi. Une approche avec un développement natif sur STM32 aurait pu éviter cette étape de transposition, malgré la meilleure facilité des outils qu'offre MATLAB.
- **Interface utilisateur** : L'interface pourrait bénéficier d'une navigation plus intuitive, notamment en raison des contraintes imposées par l'utilisation des trois boutons physiques : un écran tactile aurait pu être une meilleure solution.
- **Ecran e-Ink** : L'écran utilisé a un taux de rafraîchissement très faible (2 secondes pour un rafraîchissement complet) et une résolution très basse (200x200 pixels), ce qui contraint beaucoup les possibilités de réalisation d'interfaces (pas d'animations, certaines icônes et tailles de police sont illisibles pour l'utilisateur).
- **Stockage des données** : L'implémentation d'une solution de stockage permanent en mémoire flash permettrait de conserver l'historique des mesures même après extinction de la montre.
- **Traitement des données** : L'algorithme de détection de pas nécessitera probablement des ajustements pour s'adapter aux différentes situations de marche comme la course, la montée d'escaliers...

Bilan des objectifs atteints

Les fonctionnalités de base (affichage, interface, podomètre, capteur de pouls) sont en bonne voie, tandis que les fonctionnalités de connectivité (NFC, synchronisation Bluetooth) restent à travailler en profondeur.

Le développement itératif du châssis a porté ses fruits, nous permettant d'ajuster rapidement le design en fonction des contraintes rencontrées. De plus, l'approche de conception sur Figma avant l'implémentation sur le microcontrôleur a permis d'économiser du temps en validant d'abord les interfaces à implémenter, en prenant en compte les contraintes d'espace et de résolution imposées par l'écran e-Ink.

Les principales difficultés rencontrées concernent l'adaptation de bibliothèque pour l'écran e-Ink et l'écriture des algorithmes de traitement des données récupérées par l'accéléromètre et le capteur de pouls pour qu'ils fonctionnent efficacement sur notre microcontrôleur aux ressources limitées.

Software

Interface graphique et écran e-Ink

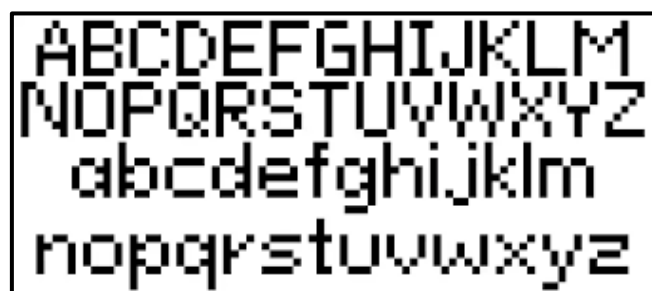
Pour gérer l'écran e-Ink, nous avons dû utiliser une autre bibliothèque que celle fournie dans l'archive du projet de base, car elle s'est avérée ne pas être compatible avec le modèle d'écran que nous utilisons. En effet, la bibliothèque fournie était optimisée pour un modèle d'écran plus grand, ce qui entraînait des problèmes d'adressage des pixels sur notre écran. Également, nous avons constaté en comparant les deux bibliothèques que les séquences d'initialisation ne correspondaient pas et provoquaient certainement le non-chargement des images sur l'écran.

La bibliothèque utilisée permet d'effectuer plusieurs opérations :

- Affichage d'une image bitmap sur 200x200 pixels
- Affichage de formes rudimentaires (point, ligne, rectangle, cercle)
- Affichage de texte et d'horloge

Adaptation d'une police d'écriture pour microcontrôleur

Une police d'écriture déclinée en plusieurs tailles est fournie dans la bibliothèque constructeur, cependant, nous avons choisi d'utiliser une autre police d'écriture dans un souci de design : Pixeloid Sans.



Les microcontrôleurs comme le STM32 ont des ressources limitées et ne peuvent pas directement utiliser les fichiers de polices d'écriture TTF. Pour les écrans e-Ink, fonctionnant la plupart du temps en monochrome, une représentation bitmap des caractères est plus adaptée.

Une police d'écriture est définie par une paire de fichiers .c, contenant les bitmaps de chaque caractère et .h, contenant les structures pour utiliser la police. Pour convertir une police d'écriture dans ce format « microcontrôleur », on utilise le programme [font2bytes](#) : ce programme permet de choisir quels caractères convertir dans le format « microcontrôleur » et la taille de la police d'écriture.

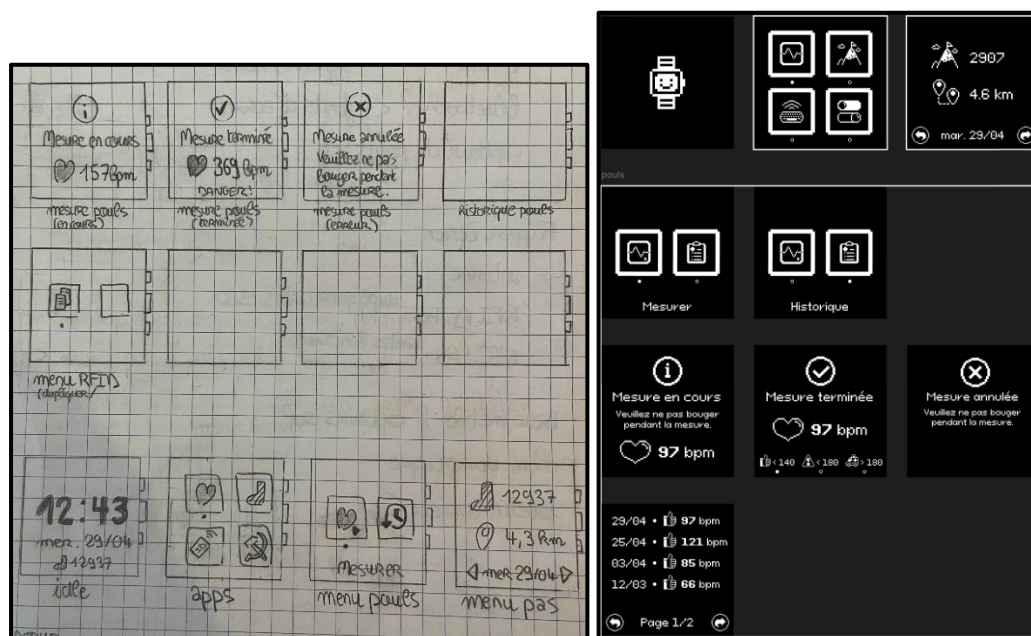
Nous avons pour le moment créé 9 tailles de la police d'écriture Pixeloid Sans, qui serviront au fur et à mesure de la conception de l'interface graphique.

Conversion d'icône en format bitmap

Pour compléter l'interface graphique de notre montre, nous intégrons plusieurs icônes tirées d'un [pack d'icônes libre de droits](#), afin d'améliorer l'expérience utilisateur. Comme pour les polices d'écriture, les contraintes de mémoire du microcontrôleur et les spécificités de l'écran e-Ink nécessitent une conversion des icônes en format bitmap lisible par le microcontrôleur (tableau de bits .c). Nous utilisons le programme [LCD Image Converter](#) pour la conversion, permettant de créer des structures adaptées. Pour le moment, 10 icônes ont été converties en un format lisible pour le microcontrôleur.

Design d'interface utilisateur avec Figma

Afin de concevoir une interface utilisateur cohérente et ergonomique, nous avons réalisé un travail de design UI sur Figma. Après avoir réalisé quelques croquis papier, cette étape s'est avérée cruciale pour visualiser et améliorer rapidement l'expérience utilisateur avant l'implémentation sur le microcontrôleur.



Croquis papier et extrait des interfaces en cours de création

Le design a été réalisé en respectant les contraintes techniques de notre écran e-Ink : une résolution de 200x200 pixels et un affichage monochrome. Nous avons également intégré la police **Pixeloid Sans**, ainsi que toutes les icônes sélectionnées pour obtenir une représentation fidèle du rendu final.

L'interface a été structurée autour de différents écrans principaux : écran de veille avec affichage de l'heure et affichage des pas, menu principal des applications, menu d'activité physique, menu d'activité cardiaque, menu NFC et paramètres. La faible résolution nous impose de devoir grossir les visuels qu'on désire implémenter dans un souci de lisibilité des informations. Les visuels et interactions ont été simplifiés pour s'adapter au faible espace disponible et dans un souci de simplicité.

Traitement des données de l'accéléromètre

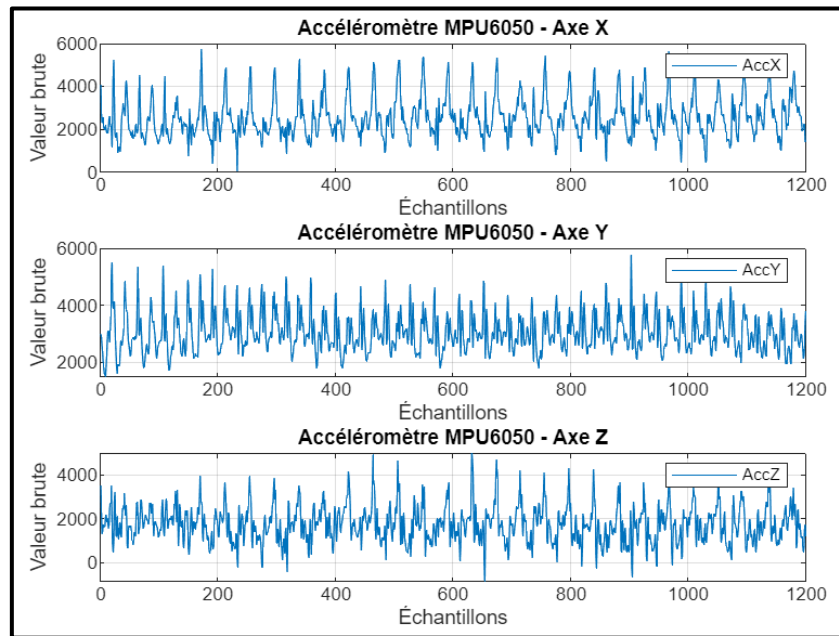
Afin de traiter et d'exploiter efficacement les données issues de l'accéléromètre MPU6050 de notre montre, nous avons choisi d'utiliser MATLAB, il offre une large gamme de fonctions dédiées à l'analyse du signal et adapté à notre besoin de détection de pas à partir de mesures d'accélération.

Acquisition des données de l'accéléromètre

Pour acquérir des données précises, nous avons en premier lieu défini le nombre de valeurs à enregistrer ainsi que la fréquence d'échantillonnage des signaux. Nous avons opté pour 1200 valeurs sur une durée de 30 secondes pour chacun des trois canaux de données de l'accéléromètre, chaque valeur étant codée sur 16 bits. Pour cela, nous avons adapté un exemple de démonstration fourni avec le module MPU6050 afin de générer une sortie au format CSV sur le port série du STM32, exploitable ensuite dans Excel ou MATLAB. Pour le traitement des données, nous avons privilégié l'utilisation de MATLAB.

Visualisation des axes bruts de l'accéléromètre

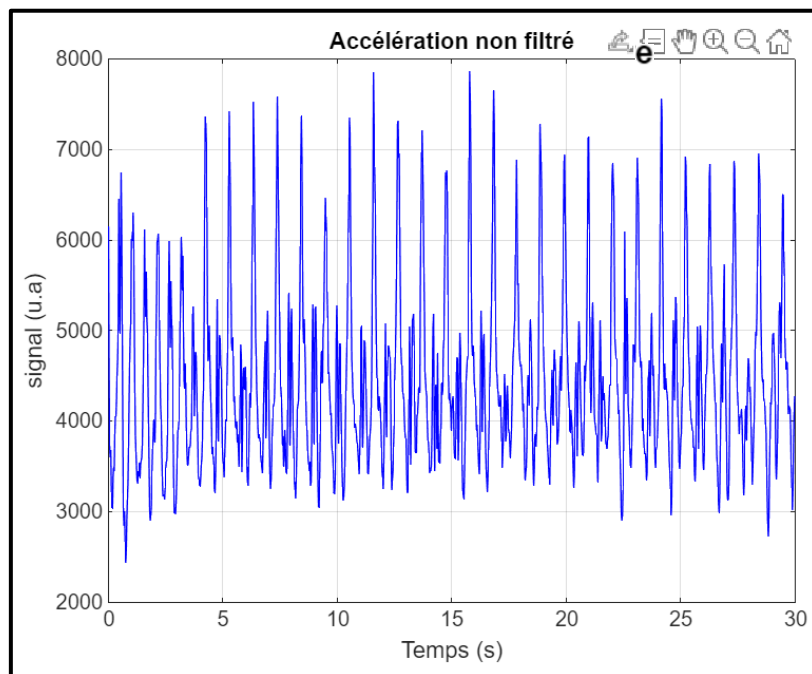
Cette figure ci-dessous montre les courbes temporelles des composantes X, Y et Z de l'accélération mesurée par le MPU6050. Elle permet de visualiser les mouvements sur chaque axe séparément et d'identifier les modèles liés à la marche. Durant les tests, nous avons effectué une marche naturelle, c'est-à-dire à une fréquence de pas régulière et normale.



Composantes de l'accélération

Norme de l'accélération non filtrée

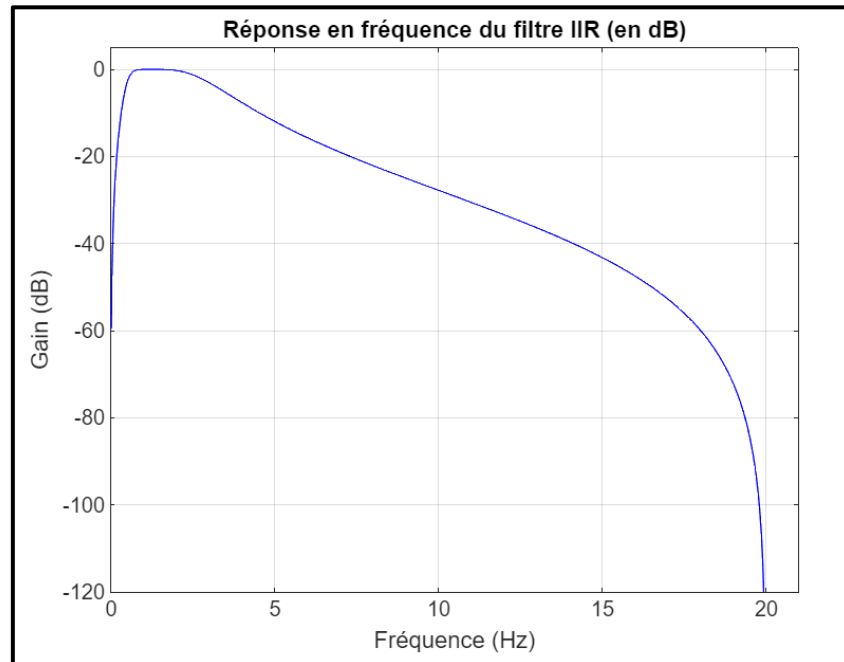
Ce graphique ci-dessous affiche la norme de l'accélération (racine carrée de la somme des carrés des trois axes). Il fournit une mesure globale de l'intensité du mouvement, indépendamment de l'orientation du capteur.



Accélération non filtrée

Réponse en fréquence du filtre passe-bande

Ce graphe ci-dessous présente le gain du filtre passe-bande (0.5–3 Hz) en non normalisé, en fonction de la fréquence. Le filtre est conçu pour ne conserver que les fréquences associées à la marche humaine, tout en atténuant les autres bruits.



Réponse en fréquence du filtre passe-bande

Diagramme pôle-zéro du filtre

Ce diagramme ci-dessous permet de vérifier la stabilité du filtre IIR utilisé. Tous les pôles sont à l'intérieur du cercle unité, ce qui confirme que le filtre est stable et adapté à une utilisation dans un système embarqué.

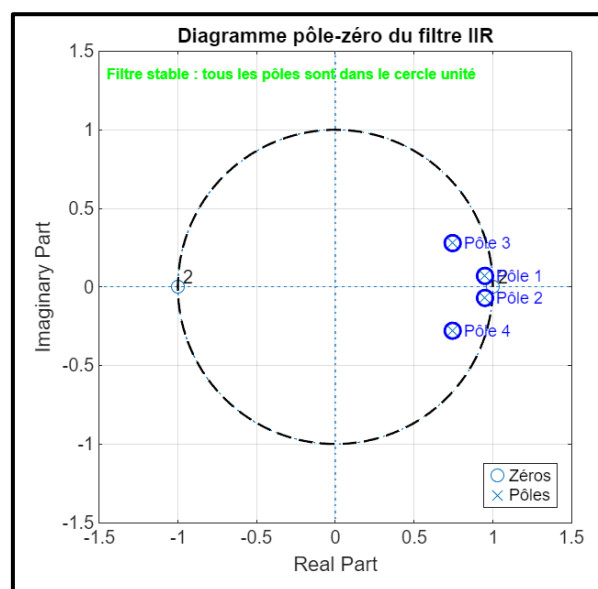
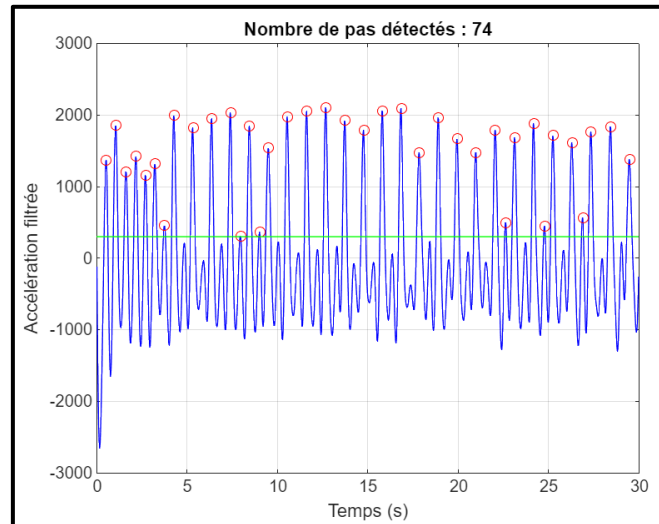


Diagramme pôle-zéro du filtre IIR

Signal filtré et détection des pas

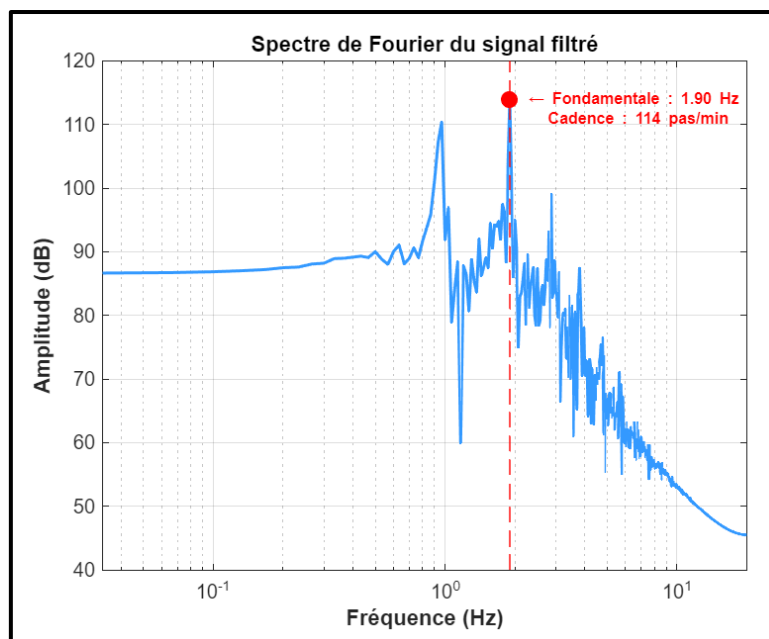
Ce graphique ci-dessous montre le signal d'accélération normée après filtrage, avec en surimpression les pics détectés (en rouge), correspondant aux pas. La ligne verte représente le seuil de détection. Le nombre total de pas détectés est indiqué dans le titre. Il correspond bien au nombre de pas comptés manuellement lors de la mesure.



Détection des pas

Spectre de Fourier du signal filtré

Ce spectre ci-dessous met en évidence les composantes fréquentielles du signal filtré. Une annotation marque la fréquence fondamentale dominante, qui correspond au rythme de la marche. Cette fréquence est utilisée pour estimer la cadence en pas par minute.



Spectre de Fourier du signal filtré

Essentiel à retenir pour le podomètre sur MATLAB

L'analyse fréquentielle du signal d'accélération à l'aide de la FFT permet de confirmer la cohérence des pas détectés en domaine temporel. En effet, la fréquence fondamentale extraite du spectre correspond directement au rythme de la marche : si des pas sont réellement présents, une composante dominante claire apparaît dans la bande 0.5–3 Hz, fréquences typiques de la marche.

Conception assistée par ordinateur d'un châssis (CAO)

Pour créer un boîtier fonctionnel pour notre montre connectée, nous avons entrepris l'utilisation de conception assistée par ordinateur (CAO). Ce châssis vise à protéger le circuit imprimé de l'écran e-Ink et l'écran lui-même, mais aussi à offrir la meilleure expérience utilisateur possible.

Nous avons utilisé le logiciel Fusion pour modéliser l'ensemble du châssis en 3D, après avoir mesuré les dimensions de l'écran. Le design se compose de deux parties : la partie principale qui accueille le circuit de l'écran et un capot inférieur pour cacher le circuit. Des emplacements sont prévus pour clipser le bracelet sur les deux longueurs du boîtier. Nous prévoyons d'incorporer des emplacements pour les 3 boutons physiques.



Rendu 3D sur Fusion

Après finalisation d'une version, le modèle 3D est exporté au format STL pour permettre l'impression 3D d'un prototype. Nous avons adopté une approche de prototypage itératif : cette méthode nous a permis d'évoluer rapidement à travers plusieurs versions du châssis, en affinant progressivement sa conception pour atteindre un résultat optimal.



Itérations du prototype de boîtier

Nous en sommes actuellement à la huitième itération du prototype, chaque version ayant apporté des améliorations concrètes basées sur les tests pratiques des versions précédentes. Chaque prototype a été imprimé en remplissant le moins de matière possible, dans un souci d'économie de temps et de matériau.

Cette approche itérative nous a permis d'identifier et de résoudre des problèmes de conception qui n'auraient pas été facile de trouver lors de la phase de conception initiale, démontrant l'efficacité de la méthode de prototypage itérative pour ce projet.

Compléments envisagés

Complément obligatoire	pts	
B – Routage de PCB avec Nucleo	3	☐
B – Routage de PCB avec microcontrôleur nu en CMS	4	☒
Compléments non obligatoires		
A – Utilisation d'un analyseur logique pour déchiffrer des trames	2	☐
C – Design CAO d'un boîtier	2	☒
D – Documentation Doxygen du code source	1	☒
E – Mesure de consommation et d'énergie	2	☐
F – Enregistrement de paramètres en mémoire flash	1	☐
G – Gestion de version du code source / Git ou SVN	1	☒
T – Jeu de tests pour valider une fonctionnalité hardware ou software	1	☐
Nombre de points ciblés au maximum	8 / 6	